Student Stress Analysis using Machine Learning

This project analyzes student stress patterns using machine learning techniques.
The goal is to:

- Understand key factors contributing to stress
- Identify different types of students using clustering
- Build a predictive model for stress levels

Techniques used:

- PCA (Dimensionality Reduction)
- K-Means Clustering (Pattern Discovery)
- LASSO Regression (Prediction + Feature Selection)

Data Loading and Setup

In this section, the proper libraries were imported and the dataset was taken from Kaggle.

The data taken from *Sultanul Ovi's Student Stress Monitoring Datasets* was comprised of two main datasets:

- StressLevelDataset: structured numerical features (main dataset)
- Stress_Dataset: survey-based questions (used for reference)

In this instance,the structured dataset is used for the models.

```
# Data Handling + Visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
# Machine Learning Imports
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error, r2_score
# UI - Streamlit
# import streamlit as st
# Datasets Provided by Kaggle
import kagglehub
```

```
path = kagglehub.dataset_download("mdsultanulislamovi/student-stress-monitoring-datasets")
print(os.listdir(path))
stress_level_df = pd.read_csv(os.path.join(path, "StressLevelDataset.csv"))

print("Loaded shapes:")
print("StressLevel:", stress_level_df.shape)
print("Loaded columns:")
print(stress_level_df.columns)
```

```
Downloading to /root/.cache/kagglehub/datasets/mdsultanulislamovi/student-stress-monitoring-datasets/1.archive...
100%|██████████| 23.8k/23.8k [00:00<00:00, 10.9MB/s]Extracting files...
['Stress_Dataset.csv', 'StressLevelDataset.csv']
Loaded shapes:
StressLevel: (1100, 21)
Loaded columns:
Index(['anxiety_level', 'self_esteem', 'mental_health_history', 'depression',
       'headache', 'blood_pressure', 'sleep_quality', 'breathing_problem',
       'noise_level', 'living_conditions', 'safety', 'basic_needs',
       'academic_performance', 'study_load', 'teacher_student_relationship',
       'future_career_concerns', 'social_support', 'peer_pressure',
       'extracurricular_activities', 'bullying', 'stress_level'],
      dtype='object')
```

Data Cleaning

Before applying machine learning, the dataset is cleaned through:

- Standardizing column names
- Removing duplicate rows
- Removing duplicate columns

This ensures consistency and avoids errors during modeling.

```
# helper function to clean the stress level dataset
def clean_cols(df):
    df.columns = df.columns.str.lower().str.strip().str.replace(" ", "_")
    return df

stress_level_df = clean_cols(stress_level_df)
stress_level_clean = stress_level_df.drop_duplicates().copy()

print("\nAfter duplicate removal:")
print("StressLevel:", stress_level_clean.shape)
print("\nColumns (StressLevel):", list(stress_level_clean.columns))
```

```
After duplicate removal:
StressLevel: (1100, 21)

Columns (StressLevel): ['anxiety_level', 'self_esteem', 'mental_health_history', 'depression', 'headache', 'blood_pressure', 'sl
```

Exploratory Data Analysis (EDA)

A quick EDA is performed on the dataset to understand:

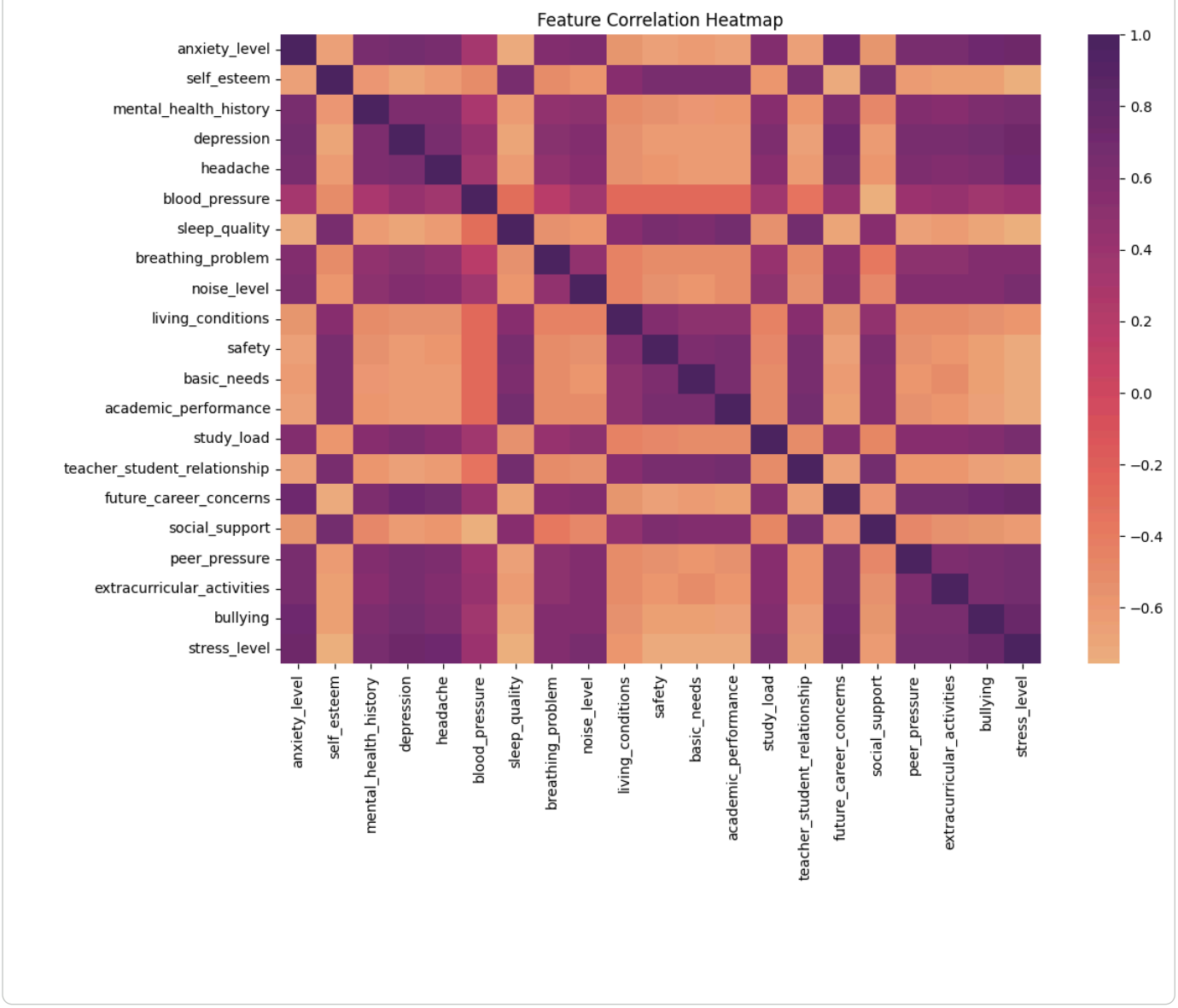
- Distribution of stress levels via countplot
- Relationships between features via heat map

This helps identify patterns and validates that the data is suitable for modeling.

```
#quick EDA of stress_level_clean dataset
# Distribution of stress
sns.countplot(x="stress_level", data=stress_level_clean, color="purple")
plt.title("Stress Level Distribution")
plt.show()
```


```
# Correlation heatmap
plt.figure(figsize=(12,8))
sns.heatmap(stress_level_clean.corr(), cmap="flare")
```

```
plt.title("Feature Correlation Heatmap")
plt.show()
```



Feature Selection and Standardization

The stress level column is dropped from the main dataframe to create:

- X: input features (habits, behaviors)
- y: target variable (stress level)

Then the StandardScaler is applied to X in order to normalize all features.

This step is critical because PCA and clustering depend on feature scale.

```
#feature selection and standardization
X = stress_level_clean.drop("stress_level", axis=1)
y = stress_level_clean["stress_level"]
scaler = StandardScaler()
scaled = scaler.fit_transform(X)
```

Principal Component Analysis (PCA)

PCA reduces the number of features while preserving most of the information.

The parameter n_components is set to 0.9 to retain 90% of the variance.

This helps to:

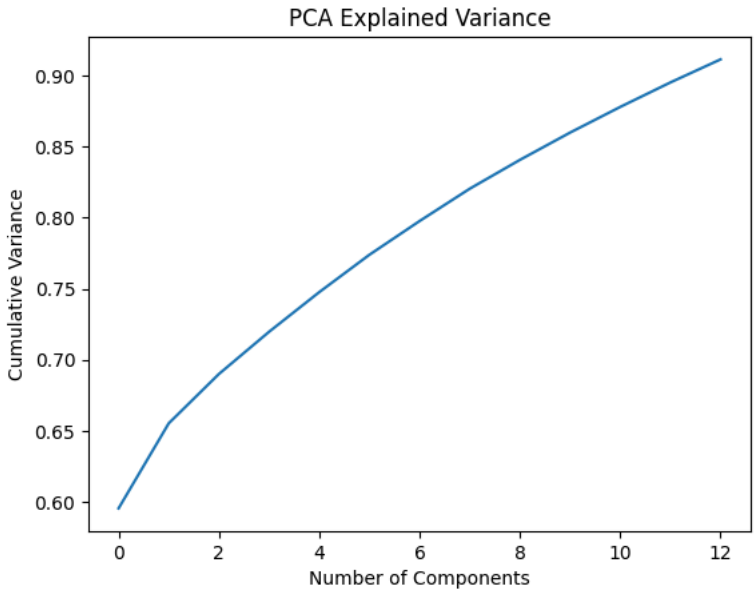
- Removes noise
- Improves clustering performance
- Simplifies the dataset

PCA Explained Variance

This plot shows how much information is retained as we increase components.

```
# PCA
pca = PCA (n_components=0.9) # helps to maintain high variance in the data, reduce noise and features used
X_pca = pca.fit_transform(scaled)

print("Original shape:", scaled.shape)
print("Reduced shape:", X_pca.shape)

# Graph helps to show how much variance (information) is captured as the number of principal components increases.
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Variance")
plt.title("PCA Explained Variance")
plt.show()
```

```
Original shape: (1100, 20)
Reduced shape: (1100, 13)
```

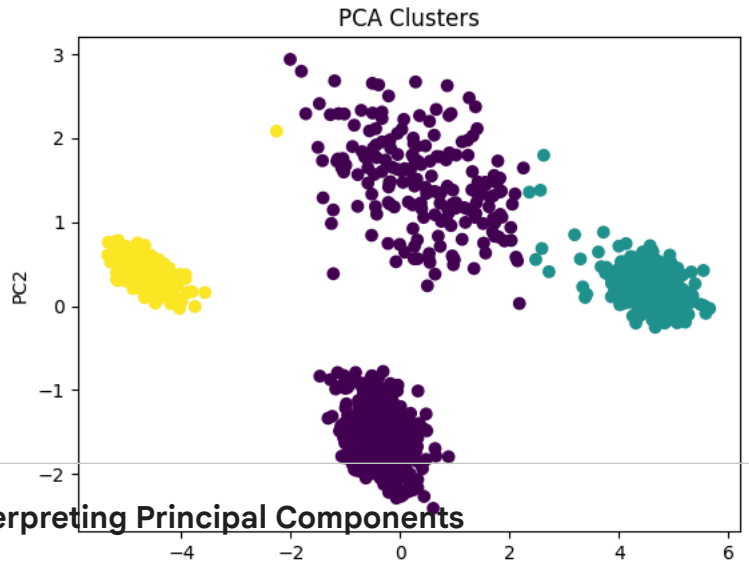


Initial Clustering (K=3)

As a baseline, the KMeans model is applied using 3 clusters. This helps to identify general groupings of students based on their behavior patterns.

```
# clustering and student profiles
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_pca)
stress_level_clean["cluster"] = clusters
print(stress_level_clean.groupby("cluster").mean())

plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap="viridis")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Clusters")
plt.show()
```

```
         anxiety_level  self_esteem  mental_health_history  depression  \
cluster
0            10.940574    18.131148               0.477459   12.172131
1            17.916399     7.871383               0.993569   21.270096
2             4.182724    27.438538               0.000000    4.172757

         headache  blood_pressure  sleep_quality  breathing_problem  \
cluster
0        2.543033        1.770492       2.581967           2.764344
1        3.913183        3.000000       1.012862           3.935691
2        1.000000        2.003322       4.488372           1.514950

         noise_level  living_conditions  ...  basic_needs  \
cluster                                 ...
0           2.516393           2.518443  ...     2.487705
1           4.000000           1.536977  ...     1.533762
2           1.468439           3.531561  ...     4.514950

         academic_performance  study_load  teacher_student_relationship  \
cluster
0                    2.506148    2.471311                      2.282787
1                    1.514469    3.958199                      1.463023
2                    4.504983    1.485050                      4.465116

         future_career_concerns  social_support  peer_pressure  \
cluster
0                      2.506148        1.770492       2.364754
1                      4.466238        0.980707       4.485531
2                      1.003322        2.993355       1.524917

         extracurricular_activities  bullying  stress_level
cluster
0                          2.502049  2.456967      0.993852
1                          4.408360  4.427653      1.964630
2                          1.501661  1.006645      0.000000

[3 rows x 21 columns]
```



Interpreting Principal Components

Principal components are linear combinations of the original features. However, on their own, they do not have an intuitive or direct meaning.

To interpret them, **PCA loadings** are used. Loadings represent the coefficients of each original variable in a principal component, showing how strongly each feature contributes to that component.

- High positive values indicate a strong positive contribution
- Negative values indicate an inverse relationship

By examining these loadings, the data can be assigned real-world meaning to each component. For example:

- PC1 may represent overall stress intensity
- PC2 may represent different types of stress (e.g., emotional vs. physical)

Without analyzing the loadings, PCA results would remain abstract and difficult to connect to meaningful behavioral patterns.

```
loadings = pd.DataFrame(pca.components_.T, columns=[f"PC{i+1}" for i in range(pca.n_components_)], index=X.columns)
```

```
PC1_factors = pd.DataFrame(loadings["PC1"].sort_values(ascending=False))
PC2_factors = pd.DataFrame(loadings["PC2"].sort_values(ascending=False))
```

```
PC1_factors
```

	PC1
future_career_concerns	0.247480
anxiety_level	0.244210
bullying	0.242731
depression	0.242709
headache	0.229389
peer_pressure	0.226360
extracurricular_activities	0.225885
mental_health_history	0.218941
noise_level	0.208863
study_load	0.204304
breathing_problem	0.189994
blood_pressure	0.145111
living_conditions	-0.198625
social_support	-0.217857
basic_needs	-0.226792
safety	-0.229310

```
PC2_factors
```

	PC2
sleep_quality	-0.240323
blood_pressure	0.744363
academic_performance	0.135897
sleep_quality	0.135088
basic_needs	0.106257
living_conditions	0.095542
safety	0.084732
depression	0.043185
extracurricular_activities	0.031601
future_career_concerns	0.008331
study_load	0.004447
teacher_student_relationship	-0.000421
headache	-0.013205
noise_level	-0.015763
peer_pressure	-0.025252
bullying	-0.089210
anxiety_level	-0.111202
mental_health_history	-0.117788
self_esteem	-0.150199
breathing_problem	-0.299804
social_support	-0.480577

Choosing an Accurate Number of Clusters

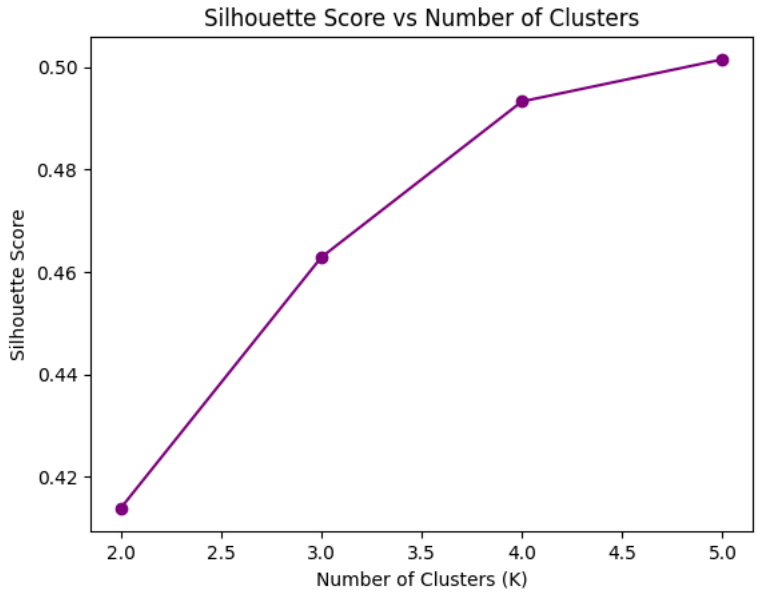
While looking at the scatterplot before, there seems to be two different groups in the purple section. To debunk whether 3 or 4 clusters should be used for more accurate data, silhouette scores were used to evaluate clustering quality for different instances of K.

This helps to determine the optimal number of clusters based on:

- Separation between groups
- Cohesion within groups

Overall, the graph highlighted a severe bend when K=4, which shows that 4 clusters are useful when analyzing this data. If K=5, it would be overfitting.

```
# in the graph i noticed that there were four clusters instead of the 3 i initially had so to compute which number is right in
from sklearn.metrics import silhouette_score

k_values = list(range(2, 6))
scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X_pca)
    score = silhouette_score(X_pca, labels)
    scores.append(score)

# Plot
plt.figure()
plt.plot(k_values, scores, marker='o', color ="purple")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Score vs Number of Clusters")
plt.show()
```



Final Clustering (K=4)

Based on silhouette scores and PCA visualization, K=4 is used instead of K=3.

This captures:

- Low stress students
- Moderate stress students
- High emotional stress students
- High general stress students

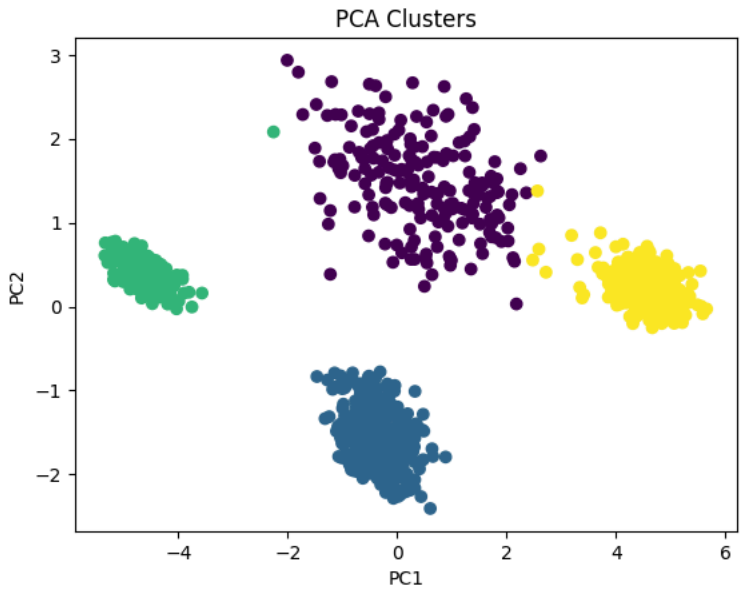
Cluster Interpretation

Clusters are labeled based on their feature averages and PCA position.

This step converts numerical clusters into meaningful categories of student behavior.

```
kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_pca)
stress_level_clean["cluster"] = clusters
cluster_df = pd.DataFrame(stress_level_clean.groupby("cluster").mean())

plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters, cmap="viridis")
plt.xlabel("PC1")
```

```
plt.ylabel("PC2")
plt.title("PCA Clusters")
plt.show()
```




```
cluster_df
```

cluster	anxiety_level	self_esteem	mental_health_history	depression	headache	blood_pressure	sleep_quality	breathing_probl
0	10.215789	15.084211	0.421053	13.042105	2.631579	3.000000	2.768421	2.35263
1	11.410000	19.980000	0.513333	11.640000	2.483333	1.000000	2.456667	3.02666
2	4.182724	27.438538	0.000000	4.172757	1.000000	2.003322	4.488372	1.51495
3	17.951456	7.883495	0.996764	21.310680	3.925566	3.000000	1.009709	3.94174

4 rows × 21 columns

```
Warning: Total number of columns (21) exceeds max_columns (20) limiting to first (20) columns.
```

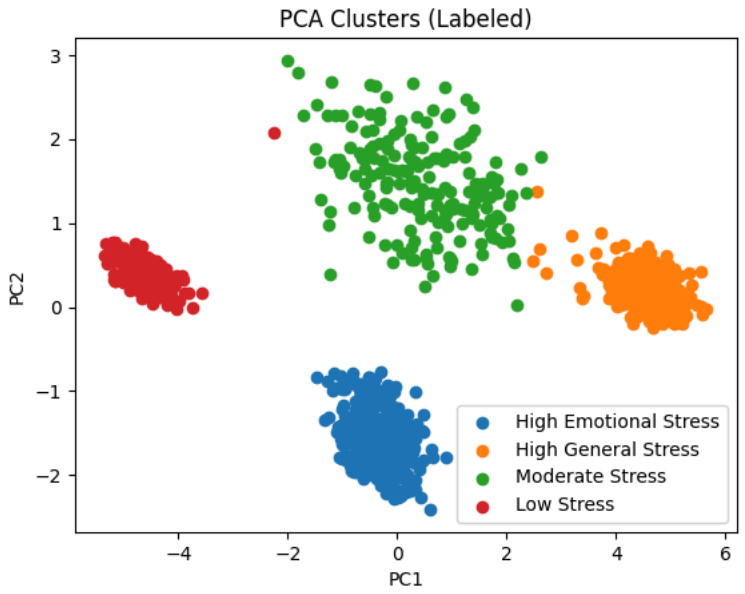
```
cluster_labels = {
    0: "Moderate Stress",        # green
    1: "High Emotional Stress",  # blue (bottom)
    2: "Low Stress",             # red (left)
    3: "High General Stress"     # orange (right) mainly about future career goals
}

stress_level_clean["cluster_label"] = stress_level_clean["cluster"].map(cluster_labels)
print(stress_level_clean.columns)
print(stress_level_clean["cluster_label"].isnull().sum())
print(stress_level_clean["cluster"].unique())
```

```
Index(['anxiety_level', 'self_esteem', 'mental_health_history', 'depression',
       'headache', 'blood_pressure', 'sleep_quality', 'breathing_problem',
       'noise_level', 'living_conditions', 'safety', 'basic_needs',
       'academic_performance', 'study_load', 'teacher_student_relationship',
       'future_career_concerns', 'social_support', 'peer_pressure',
       'extracurricular_activities', 'bullying', 'stress_level', 'cluster',
       'cluster_label'],
      dtype='object')
0
[1 3 0 2]
```

Visualizing Labeled Clusters

The clusters in PCA space are then plotted and assigned labels. This helps visualize how different student groups are distributed within the data.

```
if "cluster_label" not in stress_level_clean.columns:
    print("cluster_label column missing!")
else:
    plt.figure()

    for label in stress_level_clean["cluster_label"].dropna().unique():
        subset = stress_level_clean[stress_level_clean["cluster_label"] == label]

        plt.scatter(
            X_pca[subset.index, 0],
            X_pca[subset.index, 1],
            label=label
        )

    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("PCA Clusters (Labeled)")
    plt.legend()
    plt.show()
    # postive pc1 = anxiety, depression, workload perr pressure, negative pc1 = self esteem and social support
    # positive pc2 = physical/environmental stability, negative  = emotional and psycological stress
```



LASSO Regression (Prediction + Feature Selection)

LASSO regression is then used to:

- Predict stress levels
- Identify the most important contributing factors

LASSO automatically removes less important features by setting coefficients to zero.

The model is evaluated via:

- R^2 score (explained variance)
- RMSE (prediction error)

These metrics indicate how well the model predicts stress levels.

```
# LASSO Regression

X_train, X_test, y_train, y_test = train_test_split(
    scaled, y, test_size=0.2, random_state=42
)
```

```
lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

print("R2 Score:", r2_score(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))


lasso_df = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": lasso.coef_
})
lasso_df = lasso_df[lasso_df["Coefficient"] != 0]
lasso_df["Abs_Coefficient"] = lasso_df["Coefficient"].abs()
lasso_df["Effect"] = lasso_df["Coefficient"].apply(lambda x: "Increases Stress" if x > 0 else "Reduces Stress")
lasso_df = lasso_df.sort_values(by="Abs_Coefficient", ascending=False)
lasso_df = lasso_df.reset_index(drop = True)
```

```
R2 Score: 0.7709097801672017
RMSE: 0.39109536930323185
```

```
lasso_df
```

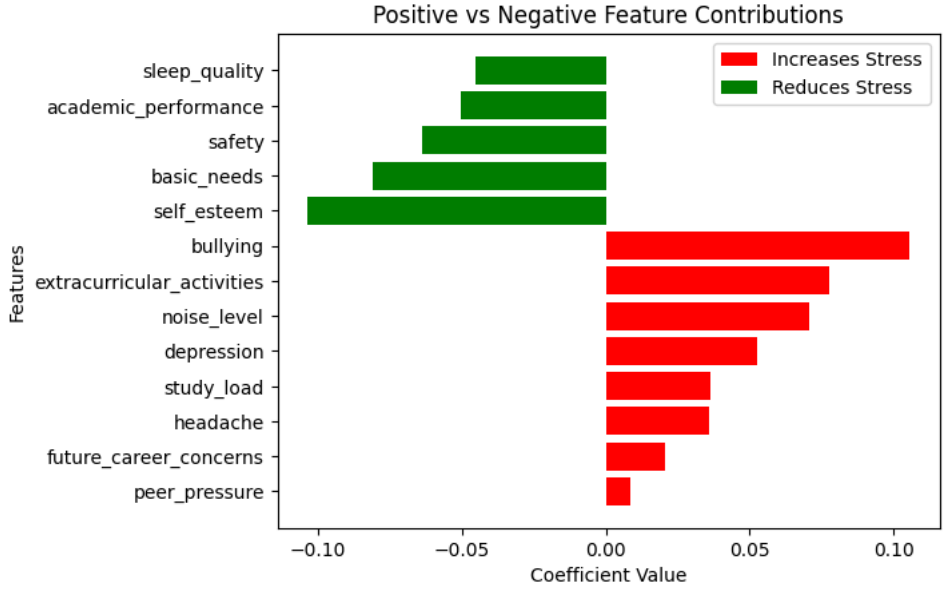
	Feature	Coefficient	Abs_Coefficient	Effect
0	bullying	0.105446	0.105446	Increases Stress
1	self_esteem	-0.103680	0.103680	Reduces Stress
2	basic_needs	-0.081085	0.081085	Reduces Stress
3	extracurricular_activities	0.077716	0.077716	Increases Stress
4	noise_level	0.070826	0.070826	Increases Stress
5	safety	-0.063878	0.063878	Reduces Stress
6	depression	0.052663	0.052663	Increases Stress
7	academic_performance	-0.050350	0.050350	Reduces Stress
8	sleep_quality	-0.045405	0.045405	Reduces Stress
9	study_load	0.036361	0.036361	Increases Stress
10	headache	0.035872	0.035872	Increases Stress
11	future_career_concerns	0.020363	0.020363	Increases Stress
12	peer_pressure	0.008741	0.008741	Increases Stress

Feature Importance Analysis

Based on the LASSO coefficients, it is shown that positive coefficients increases stress and negative coefficients reduces stress.

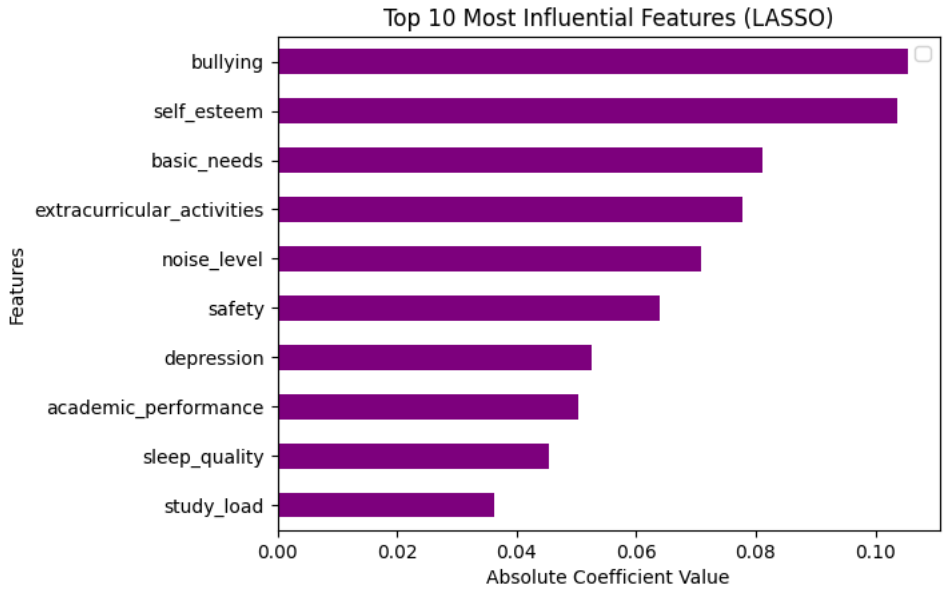
```
coef_df = pd.Series(lasso.coef_, index=X.columns)
coef_df = coef_df.sort_values()
```

```
#color coded map
pos = coef_df[coef_df > 0]
neg = coef_df[coef_df < 0]

plt.figure()
plt.barh(pos.index, pos.values, color='red', label='Increases Stress')
plt.barh(neg.index, neg.values, color='green', label='Reduces Stress')
plt.legend()

plt.title("Positive vs Negative Feature Contributions")
plt.xlabel("Coefficient Value")
plt.ylabel("Features")
plt.show()
```

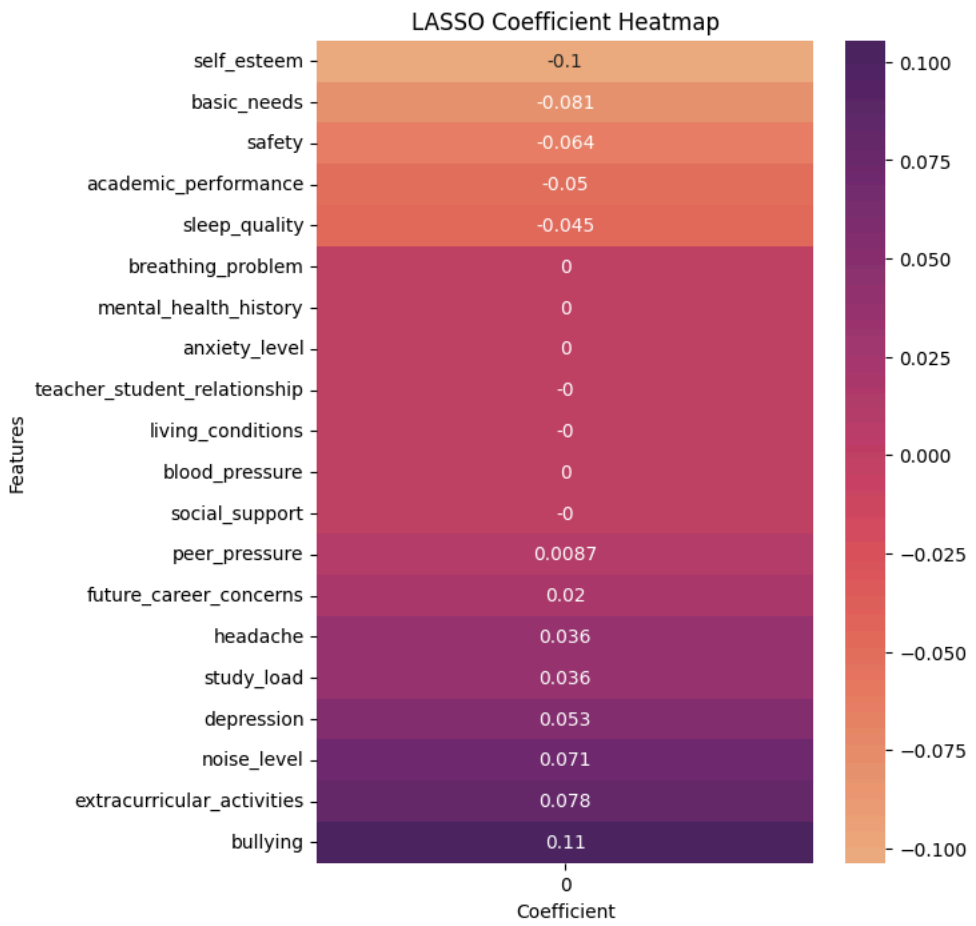



```
# top 10 features
top_features = coef_df.abs().sort_values(ascending=False).head(10)

plt.figure()
top_features.sort_values().plot(kind='barh', color='purple')
plt.legend()
plt.title("Top 10 Most Influential Features (LASSO)")
plt.xlabel("Absolute Coefficient Value")
plt.ylabel("Features")
plt.show()
```

```
/tmp/ipykernel_17332/374775281.py:6: UserWarning: No artists with labels found to put in legend.  Note that artists whose label
  plt.legend()
```




```
# heatmap
plt.figure(figsize=(6,8))
sns.heatmap(coef_df.to_frame(), annot=True, cmap="flare", center=0)
plt.title("LASSO Coefficient Heatmap")
plt.xlabel("Coefficient")
plt.ylabel("Features")
plt.show()
```



Start coding or generate with AI.